CSC8498

UrbanUI: An Environmental Dashboard

Rufus Ewings

School of Computing Science, Newcastle University, UK

Abstract

As urban populations grow, with it grows air pollution and related health threats. This project presents UrbanUI, a web application utilizing Newcastle upon Tyne's smart sensor network to generate actionable data visualisations. By using JavaScript libraries such as React and Python libraries such as Flask, an intuitive and responsive dashboard application was developed with robust data fetching aggregation, and aesthetically pleasing visualisations that place accessibility at the forefront of design. UrbanUI demonstrates how effective design and data visualisation can be used to improve public understanding and inform decision-making.

Keywords: Urban Air Quality, Environmental Data Visualisation, Smart Cities

⋆ Chief Supervisor: Dr Yinhao Li

1 Introduction

The Urban Environments that now house the majority of the world's population continue to sprawl across the planet at a rate of exponential growth (Our World in Data, 2024, urb, 2024, Korah et al., 2024, Pumain, 2012). With over 4 billion living in cities, urban environments continue to bear the impact of air pollution and its effects on public health and the environment. From asthma in children to chronic obstructive pulmonary disease (COPD) and cardiovascular diseases, the adverse health effects of air pollution are widespread and significant. Long-term exposure to air pollutants is linked to a heightened risk of diabetes, neurological disorders, and even cancer, highlighting the severe and chronic impacts on health (Manisalidis et al., 2020, Tiotiu et al., 2020). Particulate matter, particularly PM2.5 and PM10, penetrates the lungs and bloodstream, leading to systemic damage. Moreover, the detrimental effects of air pollution are not limited to physical health; the impact also reaches biodiversity, ecosystem services, and perhaps even "negatively" influences consumer behaviour (Pawanker, 2019, Cudlín et al., 2013, Li et al., 2024).

Recognising the need for innovative solutions, legislative changes and lifestyle adjustments to combat urban pollution, this project aims to empower the public with accessible and actionable environmental data. Newcastle upon Tyne houses a sensor network of over 600 strong (Acoem UK, 2024), collecting extensive real-time data on a vast number of local and environmental variables from air pressure to footfall. Public access is available to this information via the Newcastle Urban Observatory (UO) web page (Observatory, 2024) and by making use of their API, however, while their application does provide access to a huge amount of sensors, the lack of data aggregation and comprehensive visualisations makes this insufficient for public interpretation and analysis.

Co-opting the UO's slogan, "evidence for change" (Observatory, 2024), this project endeavours to deliver that evidence in a user-friendly format, empowering citizens to make informed decisions. With the help of data formatting on the edge layer and the UO's API, UrbanUI's back-end (UrbanAPI) will provide the pre-processing and forecasting needed to power a fast and responsive dashboard - before it is brought to life with React (Facebook, 2024). Despite the UO's many successes, from its API to its hardware network, the need for a more accessible and cohesive interface for public comprehension is very much present, with data often being 'locked' behind prerequisite technical knowledge to enable effective interpretation. Though users can view data from individual sensors, graphs for city-wide trends are less obvious. This being the case, UrbanUI focuses solely on citywide trends with data being aggregated behind the scenes, reducing the need for technical knowledge as much as possible, while making use of various design techniques to retain detail while reducing clutter.

The project upon which this builds (Smart City Dashboard) (Ewings, 2023) developed a prototype application with Jupyter and IPython for environmental data visualisation. The project successfully demonstrated that a dashboard could extend

the UO website's accessibility and widen the audience to which the information was available. However, this approach was limited in its accessibility for the vast majority of users, as it was constrained to running within a notebook and lacked a deliverable application. Furthermore, the application delivered data in a format poorly suited to the general public, not dissimilar to the UO's shortcoming: it functioned more as a data exploration tool for the informed than an instantly informative dashboard. Building on the previous project's foundations, this project develops a React front-end working in tandem with a Flask (Pallets, 2024) back-end, devising a more intuitive and user-friendly experience in the form of a web application. This project aims to improve data accessibility, form strong visualisations and provide a dynamic, real-time platform for users to interface with: while remaining reliable, responsive and robust.

2 Design

2.1 Overview

Successfully designing an effective user interface for a web-based application of environmental data visualisations requires extensive forethought and research. This involves examining the previous project's strengths and weaknesses, researching that which constitutes a successful dashboard from a data visualisation perspective, and careful consideration of what makes a web application intuitive in the first place. This section evaluates the previous project, establishes the explicit objectives of UrbanUI's design, describes the thorough research conducted, and concludes with the implementation approach.

2.2 Smart City Dashboard: Successes and Failures

Evaluating the successes and failures of the previous project, the Smart City Dashboard (SCD), is vital to understanding the foundations upon which UrbanUI is built. Granting clarity of insight into that which constituted a successful prototype, and the aspects of the design that lacked as compared to a fully-fledged public utility.

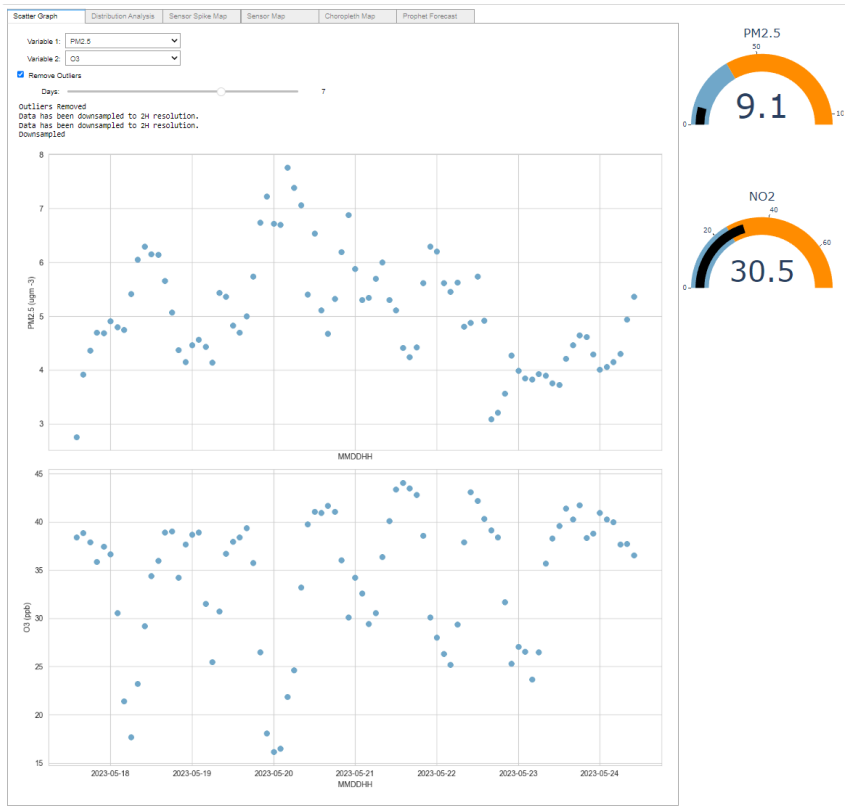

Figure 1. Smart City Dashboard: the landing page.

Strengths

An example of a success is the implementation of data fetching from UO's RESTful API. Requests are effective and can be easily adjusted through the tool, dynamically reacting to the requirements of the user and caching the most recently fetched data in local storage to ensure data persistence in the event of somewhat frequent UO downtime - and near real-time updates the rest of the time. The SCD provides a range of comprehensive visualisations, building upon the UO website with such implementations as the spike and choropleth maps, which provide a clear and easily understandable representation of location-based pollution data. These should be considered when creating UrbanUI to maintain engagement through interesting yet pragmatic visualisations.

Basic forecasting capabilities were also implemented, drawing on the success of the UK Defra Air website (Air, 2024) and enabling early warning signs of dangerous pollution to a reasonable degree of accuracy. The forecasting model achieved a mean absolute error (MAE) of 14.82, although this is relatively high compared to more advanced models (Zhang et al., 2024), still marks a success in terms of this prototype and could be expanded upon through the implementation of more sophisticated tools than prophet (Facebook), and increased use of hyper-parameters for fine-tuning.

Tabs to switch between elements were intuitive, however, switching between different graphs this way signposted it as more of a tool than a dashboard and could be improved upon. In addition, the ability to compare aggregate data of different variable types with ease was a great success but the implementation solidifies its place as a data exploration tool over a dashboard. The implementation of slider and checkbox widgets to provide control over the period shown and the inclusion of outliers, backed up by fast reload times, made the tool feel both powerful and responsive. The key air pollution variables were identified and presented. PM2.5, PM10 and NO2 remain an effective representation of some of the pressing threats to public health in urban environments. Gauges are present on the front of the app to give immediate feedback for the last 24 hours, turning from blue to orange if this exceeds the healthy limits according to UK government figures (Department for Environmental Food and Rural Affairs, 2024).

Weaknesses

Conversely, however, the SCD isn't without its flaws. The application is limited in accessibility, as it requires users to operate within a Jupyter Notebook environment and lacks a deliverable web application. This is arguably the primary consideration when identifying this tool as a prototype rather than a finished deliverable.

What restricts its accessibility further is that the format was more suited to data exploration for informed users rather than providing an instantly informative dashboard for the general public. This is the case in that interpreting data requires navigating through tabs to look at different graphs. Though the division into tabs enables straightforward comparison between variables, with multiple graphs of the

same type per page, this detracts from the concept of the dashboard delivering all the necessary information as clearly as possible. This is similar to the Bloomberg Terminal, a dashboard of sorts which excels at "concealing complexity" and focuses on "designing with empathy" (Bloomberg, 2022).

The SCD lacks interactivity such as tool-tips and animations which would improve the visual appeal and make the application feel more dynamic. More visually appealing elements would increase user engagement and improve comprehension by the public. Similarly, the generated graphs, while informative, varied in complexity, making some of them difficult for non-technical users to understand, while lacking additional information. This could be improved upon with the implementation of an information tab, outfitted with links for where the user can go to find out more, further improving accessibility. An alternative solution would be an 'i' icon outfitted with a tool-tip, on each of the graphs and variables.

2.3 Design Objectives

Before progressing it is important to outline the core design objectives of UrbanUI.

Performance:

UrbanUI strives to deliver a high-performance experience, handling even larger datasets quickly and efficiently. This requires a joint effort from the front and back ends to ensure no elements are making unnecessary calls or processing ineffectively. Central caching should be implemented to ensure multiple graph elements share resources and load in unison. Users should not be made to wait for particular components and lengthy processing tasks should be deferred where possible, for example, forecasting should not be displayed on the landing page. Lazy loading should also be implemented where possible to defer processing if it provides a smoother experience.

Security:

Online deliverables should be robust and meet all the necessary security prerequisites that make up the modern web. Any data storage should be handled securely and per privacy regulations and the GDPR if relevant. HTTPS should be utilised for all communications and care should be taken during development to ensure no irrelevant or unnecessary data is published.

Accessibility Features:

Both interface and graphs should be implemented with colours suitable to those with colour-vision deficiency. Implementation of dark mode can improve the accessibility of the application, reducing eye-strain (Erickson et al., 2020), as well as improving battery use on devices with OLED screens (Eisfeld and Kristallovich, 2020) and making the utility feel more modern. Thus, a dark-mode toggle should be implemented to ensure the site aligns with the rest of the modern web (Ma et al., 2023) and libraries which can query the device's dark-mode preferences should also be capitalised upon. With accessibility of information being a key aspect of this

program, users of varying levels of education should be encouraged to interpret and analyse data. Steps should be taken to prevent the dashboard appearing too complex and graphs of different complexity should be available, with the most straightforward at the foreground of the application. Implementation of informative tabs should be considered for education and making the visualisations understandable by all.

Intuitive Interface:

Create an interface that is easy to navigate, regardless of technical literacy level. This can be achieved by following set precedents across the web, as well as by drawing inspiration from both digital examples and the real world. On top of this, the lack of need for great amounts of technical know-how is a key element of how UrbanUI will differ from other dashboards (Jin et al., 2021). The target audience is not just academics and professionals, but also those of all disciplines who otherwise are discriminated against via failure to provide accessibility of information.

Interactivity:

Implementation of UI to cycle between variable types, with or without outliers, should be present to provide an engaging user experience, so long as these do not interfere with the readability and accessibility of the dashboard.

Mobile Responsive:

The dashboard should be fully functional on mobile devices. With the increasing use of smartphones and tablets for accessing the web, responsive web design is crucial for ensuring a seamless user experience across a variety of platforms (Rashid, 2017).

Feedback:

As part of the ongoing design process, a feedback mechanism should be considered for future development. This could take the form of a support email address, an FAQ, or a feedback tab. As stated above, any user input data should correspond to the relevant regulations - and any data storage should be implemented with the utmost caution. Personally identifiable information should be kept away from the page, and as such a support email would require setting up a new address.

2.4 Research and Best Practices

Data-ink Ratio:

The data-ink ratio is frequently referenced as a "popular measure of graphical information content, where the "ink" related to data is divided by the total amount of "ink" in the graph". A medium data-ink ratio should be considered optimal for this application to ensure the balance between user comprehension and information density is maintained (McGurgan et al., 2021).

Drill-down Features:

One effective method of preventing information overload in a dashboard is through the use of 'drill-down features', granting greater control over data visualisations and hiding complexity until it is called upon, putting power over the dashboard in the hands of the users. (Yigitbasioglu and Velcu, 2012)

Dashboard Design Tips:

Pastushenko et al. (2019) provide several tips for effective dashboard design:

- Eliminate non-data pixels (decorations) to decrease the distraction of users.
- Consider Gestalt principles to help users recognize coherent groups of UI elements better.
- Select appropriate charts and colours to emphasize the relationship between data and highlight critical information.

Gestalt Principles:

The Gestalt principles are a list of "rules of the organization of perceptual scenes" (Todorovic, 2008) which should be consulted when designing the effective organisation of visuals. Some key elements include:

- Figure-Ground Articulation: elements in the foreground will be considered separate from those in the background.
- Proximity: elements near to one another are considered related.
- Similarity: elements that look similar are perceived to be more related and should be considered when selecting colour schemes.
- Closure: The mind tends to fill in missing information to create a whole, for example, missing data points.
- Symmetry: people prefer symmetrical and organised structures, and these will generally be considered more aesthetically pleasing.

Regarding symmetry, Pastushenko adds, "We did not detect any correlation between the perceived symmetry and overall first impression," meaning symmetry need not be enforced unnecessarily, however, according to the Gestalt principles, balance is often preferred aesthetically (Pastushenko et al., 2019, Todorovic, 2008) and as such the dashboard must place a moderate level of importance on the balance of elements, particularly on the landing page.

2.5 User Interface (UI)

The layout will provide a clear visual hierarchy, with the most critical information accessible at the top. The use of contrasting colours, varying text sizes and strategic placement of elements will guide the user's attention while striving to prevent clutter. To ensure individual graphs are accessible to all users, for example, those with colour-vision deficiency, the user interface which combines these graphs should follow a similar philosophy. Further, the arrangement of these dynamic graph items should promote ease of use and cultivate a cohesive and pleasant user experience.

MaterialUI

MaterialUI (MUI) (mat) was selected as the primary library to develop a modern and responsive user interface. Implementing Google's 'Material Design', MUI provides not only a clean interface but due to its ubiquitous use across the modern web, an intuitive one. This was selected over Bootstrap (Bootstrap Contributors, 2024) due to its comprehensive set of available pre-built UI components and extensive styling options, ensuring a cohesive and familiar look, and working towards the goal of creating a visually appealing interface.

Colour Schemes

Colour schemes suitable for those with colour-vision deficiency, the most common deficiency being red-green (National Eye Institute, Accessed: 2024), are essential. D3 provides a vast array of colour schemes for use in graphs and charts. Color brewer (ColorBrewer) is cited as a useful resource for selecting effective colour schemes, and hosting qualitative, sequential, and diverging colour maps (Moreland, 2016). Diverging colour maps are perceptually uniform, keeping the colour change consistent for ease of data interpretation (Moreland, 2016), and especially in the case of air pollution will highlight the highs and lows of the data across the graph. In addition, diverging colour maps aim to be inclusive for those with colour blindness, whereas those such as rainbow maps fail (Roelandts, 2020, Ware et al., 2023).

Another consideration when selecting colour schemes for graphs is that perceptually "symbolic risk colours (e.g., red conveys warning) may influence risk beliefs and ambiguity and exert more influence for larger than smaller units of perception" (Severtson and Myers, 2012). As such, to convey danger successfully, more 'urgent' colours should be selected, such as orange/yellow/purple over light blue or green.

Interactive Elements

Interactive elements, such as tooltips and checkboxes, should be implemented to provide effective 'drill-down' information and create a more intuitive and engaging user experience. These interactions will aim to be subtle yet informative, providing additional context without overwhelming the user. Further, separation by tab can grant the user greater control over the data being displayed on the screen, such as categories of variables and options to alter the number of graphs being displayed.

2.6 User Experience (UX)

To effectively understand user needs, extensive research must be undertaken, ideally in the form of iterative user testing and surveys. For the example of UrbanUI, the evaluation of the SCD will be thoroughly considered throughout all stages of development, and the target audience should be at the forefront of all design choices. Differentiating from the UX of the UO portal, this project works toward the aim of a more 'dashboard-like experience by compiling a series of graphs into a single-screen design. A strong consideration while doing this, however, should be to prevent

the 'information overload' (Yigitbasioglu and Velcu, 2012) and following the advice of these papers to provide alternative viewing options to grant users control over this view of data, such as "drill down features." Visualisations should be clear and intuitive, first and foremost. Dynamic charts and maps provide control over visualised data and greater depth without hindering clarity.

Consistency and Familiarity

Pages across the site should have an air of familiarity and should be consistent not only with one another but even across the web, wherever possible. Playing somewhat into 'intuitiveness', familiarity is key to providing an effective user experience - without having to explain anything. Examples include the burger menu for bringing up further settings, the sun/moon toggle for light and dark mode, and the balanced grid page layout that MaterialUI produces in abundance. UrbanUI aims to be consistent across all platforms, with buttons, fonts and element spacing maintained in a familiar and yet functional way. Element sizes should dynamically adjust to fit the screen size and provide a seamless user experience.

Attention Span

In an increasingly information-dense and short-form content-focused world, the digital-native generations are noted to have a shorter attention span and to desire convenience and immediacy(Chicca and Shellenbarger, 2018). Consequently, the need for high levels of performance is greater than ever before. The snappy nature of a website is not only useful for providing a smooth experience but critical for engaging the user in a successful capacity. In addition, the need for data to be quick and easy to interpret is emphasised further. The dashboard format having many sources of information on screen simultaneously is hoped to play into keeping the attention of individuals familiar with being occupied by multiple streams of information, anecdotally cartoon sketches or app game-play alongside other short-form content (Sabieddine, 2023), or even simply the second screen viewing that comes with the popularity of both mobile phones and television(Van Cauwenberge et al., 2014).

2.7 Technical Implementation

The back end, UrbanAPI, will be developed in Python using the Flask library. Building upon the successes of the RESTful API implementation in the SCD, UrbanAPI will provide a platform for the front end to easily fetch pre-processed data from the cloud layer for immediate creation of visualisations. React was discussed in the previous dissertation as a potential solution to the aesthetic limitations brought about by IPython. Similarly, D3 (d3j) was considered as a potential upgrade to Plotly, as a powerful tool for generating dynamic charts and maps: implementing 'drill-down' tools to grant greater control over the amount of data being visualised. The increase in program complexity is to be offset by the improvement in the result. Besides this, other libraries such as ChartJS (cha) are utilised as appropriate for forecasting plots, with care taken so as not to break up the cohesion of the dashboard. With MUI providing the majority of the UI, relatively few libraries are

necessary, allowing for the bringing together of a clean, stylish and cohesive web page.

2.8 Iterative Design

The design should take an iterative process, however during initial development, in place of seeking public feedback all evaluation and review are to take place in-house through taking a step back to reconsider the core tenets of the application and how these are being met - or more critically, not met. After initial development, feedback represents a critical element of continuous improvement, informing and directing the ongoing development process. Implementation is expected to develop a functioning design of a web page, demonstrating the back-end and front-end working in unison to deliver a responsive web application.

User feedback will guide the development and also indicate the success of the project. Performance testing to ensure the speed lines up with expectations and the loading times of similar applications. Accessibility compliance can be considered by comparing the final product to the internationally recognised WCAG accessibility standards (UK Government, Accessed: 2024). After this evaluation, steps can be taken to improve user experience, program efficiency and accessibility as necessary.

3 Implementation

3.1 Overview

This section focuses on the implementation of the prospective design into a web application, adapting a back-end from the previous project and developing an entirely new front-end.

3.2 UrbanAPI

Covering back-end development first, primarily written in Python, UrbanAPI utilises the web application framework Flask to create a lightweight API for the front end to interface with. Socket IO provides real-time communication between the client and server, enabling the front-end to fetch data as required, but also providing bidirectional support, allowing for the server to communicate with the client if required. The majority of data processing is achieved with Pandas, handling outliers and erroneous data removal. Facebook's Prophet library provides robust automatic time-series forecasting.

End-point Design

Endpoints are implemented for data retrieval via `GET` requests.

- The `/api/data/<variable>` endpoint enables retrieval of the environmental data. Setting a parameter for the variable type, as well as optional parameters for the number of days, and whether outliers should be included, with the default being 1 day and no outliers.
- The `/api/forecast/<variable>` endpoint provides forecasts using the prophet model, returning the timestamps and values of the data, and a mean average error describing the accuracy of the model.
- The `/api/averages` endpoint generates mean aggregate data over the past 24 hours and requires a variable parameter.

Data Fetching and Aggregation

Data retrieval from the UO API involves fetching the raw sensor data and processing it for use in visualisations. The `AggData` class is adapted from the SCD (Ewings, 2023), taking a variable and number of days as arguments, building an API request and fetching aggregate data all within the constructor. The UO request requires a variable, aggregate method, aggregate period, start time, end time and sensor type. These are set accordingly, with the start time being set to the appropriate number of days ago, and the end time set to now.

Aggregate data was set to median at 15-minute intervals, to ensure a high level of accuracy without introducing unnecessary processing. Further control over data granularity is provided through the downsampling method, adjusting the resolution to a mean value at decreasing intervals corresponding with the number of days. This is employed liberally, ensuring visualisation clarity is maintained with increases in data. When fetching data, a backup CSV file is created and stored locally under

the variable name and number of days, overwriting any previous file with that data. If the UO is experiencing downtime, the relevant CSV file will be utilised to ensure data integrity and persistence.

Data Cleaning

As part of data cleaning, entries marked as 'suspect reading' are removed alongside all elements with missing or erroneous data entries. The interquartile range is implemented to filter out outlier readings from the data frame, ensuring only non-outlier entries between the first and third quantiles make it to the visualisation elements.

Error Handling and Robustness

Try and except blocks are used to catch key errors and index errors in the fetched data, replacing these with empty strings to ensure no exceptions interfere with the API. If the fetched data implies the site is down for maintenance, the program attempts to load one of the locally stored backup CSV files, only failing to provide data in the event one is not available. This mechanism ensures data persistence and availability during downtime of the UO website. Front-end data fetching is implemented with a central caching and queuing system, meaning requests to UrbanAPI are performed in an orderly manner to prevent overburdening the cloud layer. This is supported by a debouncing rate of 300ms to improve stability while fetching. Error boundaries are also implemented on the front end to catch and display any errors that might take place within the UI, providing clear feedback to users and facilitating effective debugging.

Forecasting

Forecasting takes place on the back end and is accessible via the forecasting endpoint, which will provide the forecast data and an estimated error margin. Facebook's Prophet model was selected for forecasting due to its automatic and robust nature. The presence of outliers or erroneous data are handled effectively and won't interfere too heavily with prediction-making. As the model is more or less 'plug and go' it is appropriate for use with this application, able to generate relatively accurate forecasts without extensive fine-tuning which would be more suited to a forecasting application such as is available on the Defra website. The model is given the past 30 days of the relevant variable's data for forecasting, which it then splits into 80% for training and 20% for test data. This is done to allow for the calculation of the mean average error between the predicted values and the values in the test set.

The forecasts on average generated a higher MAE during May 2024 than during May 2023. This could be due to changes in environmental factors, such as higher levels of air pollution than can effectively be predicted by the model. It is also possible that there could be greater levels of seasonal variation across the past month, as compared to last year. It is more likely, however, that this is a result of unstable predictions due to a lack of data. To combat this, future development should find ways of handling larger training data sets, while maintaining performance. This

could be achieved through regular server-side caching, handling larger calculations at intervals across the day for the data to be fetched by users, rather than generating forecasts on demand.

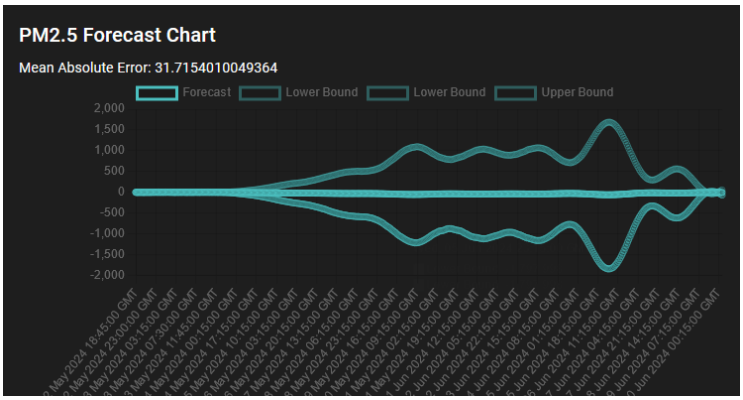

Figure 2. Forecasting: PM2.5 values, 30 days

3.3 UrbanUI

The front end makes use of the functional JavaScript library *React* for building user interfaces and web pages. Developing web pages with React removes the need for working with HTML and CSS extensively, especially when combined with MaterialUI (MUI), a library of components for fast and effective web development. The D3 library is also employed extensively to provide dynamic and interactive charts with great amounts of customisability and room for future development based on feedback. The leaflet (Leaflet.js, Accessed: 2024) library is also used to provide maps and to work with D3 for creating spike and bubble map visualisations.

Components

The `app` component is the base of the application, as the initial script called by `index.js` on launch. MUI's `CssBaseline` ensures consistent cross-browser rendering by establishing the set of CSS styles at the root that will grow across the entirety of the project, without the need for extensive CSS programming for different devices. In addition, the theme provider enables site-wide theme customisation, in this case, used to implement dark mode, setting the default to the response of a media query - `prefersDarkMode` which checks what the default setting is on the current device. The `useState` hook allows cross-component variables so that all components can follow the themes laid down here, and in this case, so that the App Bar can control elements in the dashboard component. An error boundary is also established here to catch errors and return an error message in the event of a critical error, helping debugging and giving users clear explanations in the event of a fault - these can then be used when contacting support.

The `AppBar` contains buttons for the sidebar burger menu, a timescale drop-down menu and a dark mode toggle button. In addition, the title is displayed

clearly in the top left. This layout provides a clean UI free from clutter and distractions, in line with the Gestalt principles. The burger menu in particular is a strong tool for providing extensive functionality, maintaining clarity and following the intuitive precedent laid down by vast sections of the web. Below this the `Dashboard` component is called with the relevant use states for the toggles and options passed as arguments. This hierarchy of components ensures modularity and emphasises simplicity for future development.

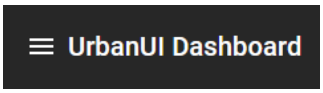

Figure 3. Burger menu

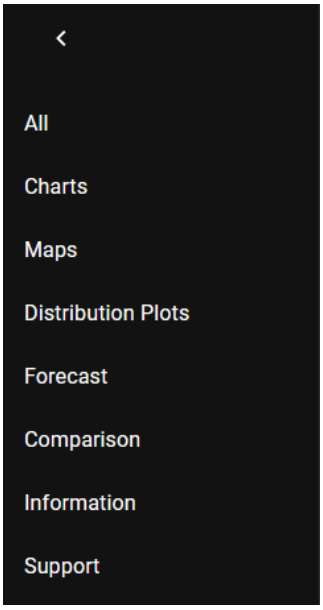

Figure 4. Side bar menu

The **dark mode** toggle emphasises the design principle of accessibility, as well as granting the user direct power over the application based on preference. The sun and moon icon is used for the logo to reduce the amount of text on the screen while retaining clarity in its meaning. Accessibility for this feature is considered in the form of alt-text so that screen reader software can understand and read out what the button does, should a user require it - although the majority of the dashboard is still ineffective at achieving this due to the complexity of outputting graphs as audio. A possibility for the future could be the use of AI to interpret graphs, before reading out their contents. The dark mode state also prompts certain visualisations to update their colour scheme to a colour scheme determined to be effective.

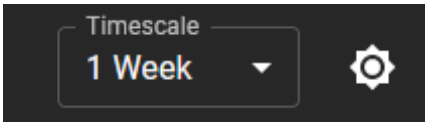

Figure 5. Dark mode toggle

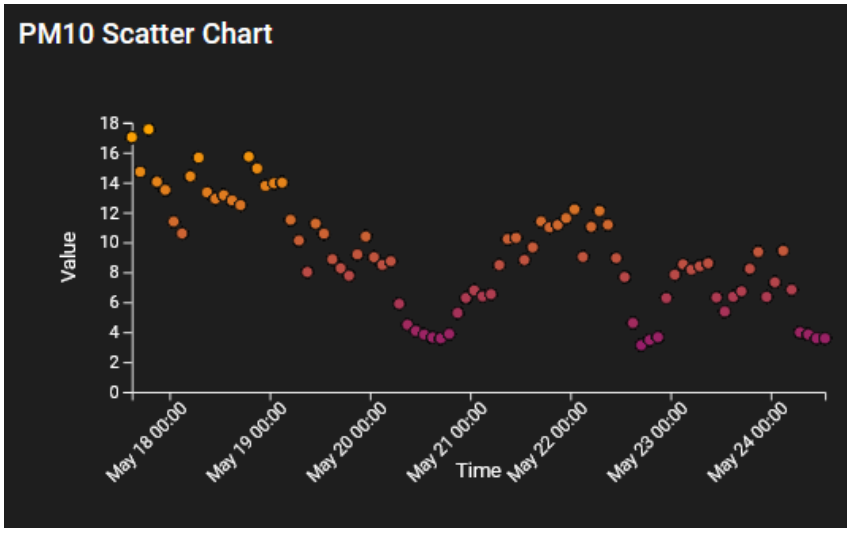

Figure 6. Scatter graph: PM2.5, one week.

The **dashboard** component represents the primary component of this application, from which all the visualisations are called. The sidebar, accessed via the burger menu button, contains the categorisation of graphs to grant extensive user control over the displayed visualisations. Defaulting to 'All', the landing page shows all visualisation charts, maps and distribution plots. Forecasting is kept in its tab separate from the 'All' page to prevent unnecessary increases in load times and to keep the site feeling 'snappy'. To make the dashboard as accessible as possible, the plots are ordered with simplicity and aesthetics in mind: Line Graph, Scatter Graph, Violin Plot, Box Plot, Spike Map and Bubble Map.

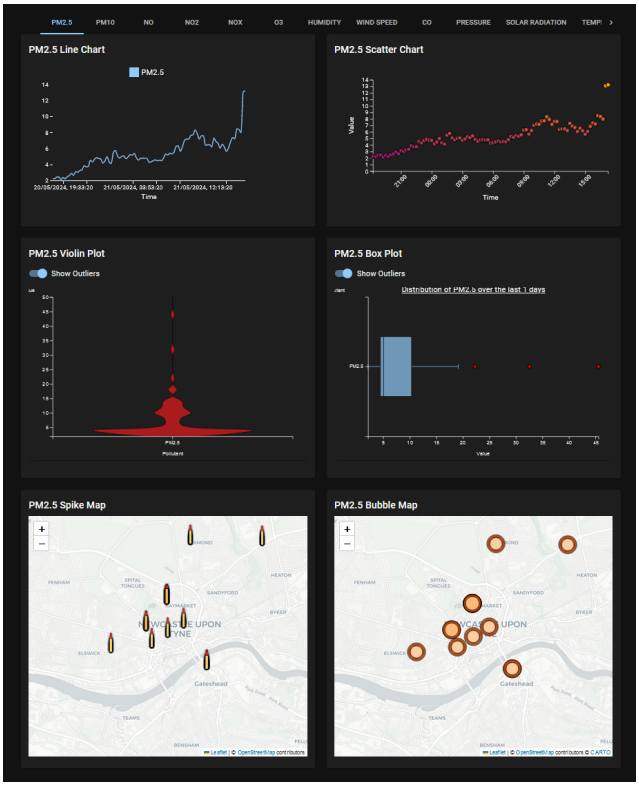

Figure 7. Main dashboard: dark mode

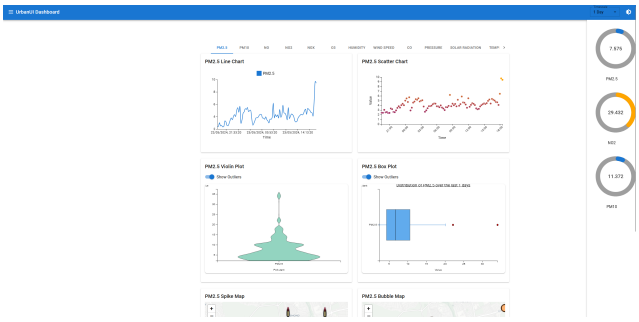

Figure 8. Landing screen: light mode

The **taskbar** contains all the variables at the top of the dashboard element. Selecting one of these variables will update the visualisations to that variable, updating the text on each visual and incurring a loading spinner while the data is fetched as visual feedback to the user that their input has been received and is being processed.

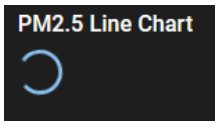

Figure 9. Loading UI element

The **comparison tab** brings up a series of checkboxes for each variable. Checking one of these boxes will display graphs for that variable, organised by graph type such that each graph is kept together for ease of comparison. This adopts the success of the easy comparisons featured in the previous project, but without having it interfere with the core functionality of the dash. For example, using the comparison tab, one could compare and identify the strong correlations present between various weather statistics, such as air pressure and temperature, and air pollutants such as PM2.5.

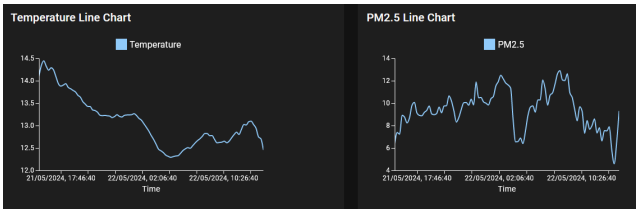

Figure 10. Line Graphs comparing temperature and PM2.5 over the past week

The **information** tab links various websites the users can go to for more information on the visualisations and variables used by the dashboard, infusing the site with a connection to the greater web and educational resources that far exceed what could be provided within the scope of the dashboard.

Figure 11. Line Graph: PM2.5 over the past 24-hours

The **support tab** provides a support email address for users to forward any queries or suggestions to. This is a key element to iterative design, providing a source of feedback for future development.

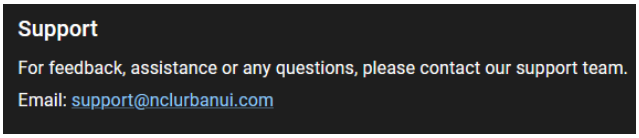

Figure 12. Support Tab

The `withAutoResize` component makes use of the `ResizeObserver` library to adapt visualisations to fit the card they are being kept in, which in turn is sized based on the screen size.

Data Visualisation

Breaking down the numerous data visualisations being employed:

- Line Graph: Demonstrates the trends of data over time in a simple-to-understand plot for visualising change or identifying patterns, such as increases in pollution on days when local football teams are playing, enabling educated decision-making on mitigation. Tool tips appear upon the event of the mouse hovering over the graph, drawing a line down the y-axis to demonstrate what is being displayed. The tooltip shows the date, time, variable and values in an intuitive layout. Implemented from MUI's charts library, MUI X, the line graph uses D3 behind the scenes with a focus on performance when working with larger amounts of data. The x-axis could be adjusted for clarity, perhaps dynamically adjusting based on the timescale. Outliers are removed and down-scaling has taken place. Linear scaling was selected over logarithmic to ensure it is easily understandable by the average user, especially since the orders of magnitude are not that great.

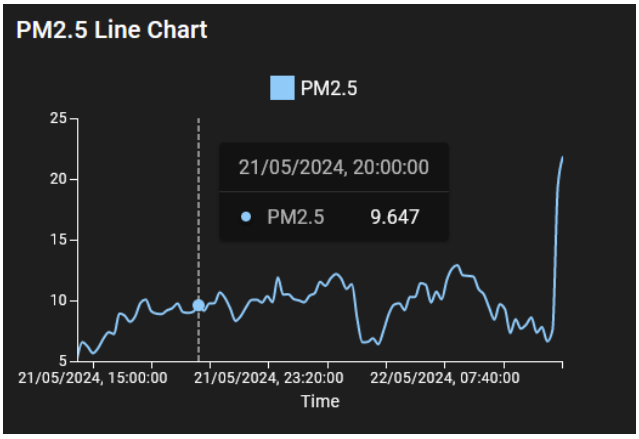

Figure 13. Line Graph: PM2.5 over the past 24-hours

- Scatter Plot: Visualises the distribution and spread of a variable over time, with hover-over tooltips for examining exact values for closer inspection. This could be further improved with the addition of an option to compare multiple variables on one plot to illustrate the relationships between them more effectively, however, this was left out in favour of the accessibility of simplicity for the time being. Implemented with D3, the x-axis scales to suit the amount of time being shown and any overlapping points are clustered together to prevent clutter.

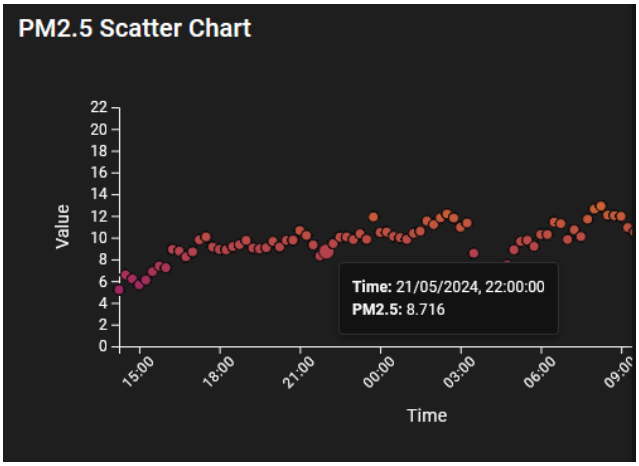

Figure 14. Scatter Plot: PM2.5 over the past 24-hours

- Violin Plot: A type of density plot, the violin plot displays the distribution density of the data over different values, demonstrating the overall shape of the data and highlighting where the majority of the data points are concentrated. This is completed with an interactive toggle switch to turn outliers on and off as required for distribution analysis. This violin forgoes markings and tooltips for interquartile range to focus on simplicity, with the box plot elements separated into their own plot. Implementation with D3 granted control over the plot to leave out markings and tone down complexity.

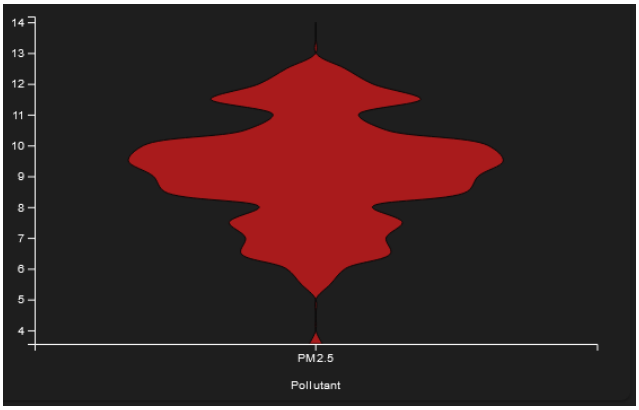

Figure 15. Violin Plot: PM2.5 over the past 24-hours, outliers-removed

- Box Plots: the interquartile range, median and outliers are all demonstrated via a box, lines and whiskers laid along the x-axis. This axis was selected as only one variable is being visualised, thus it makes sense for the plot to take up all the space available - in addition, simplicity is emphasised over the y-axis oriented plot, with the medium data-ink ratio being somewhat forgone in favour of not over-complicating a plot which may be less obvious to casual observers. The shape of the data is revealed similarly to the violin plot, though in a slightly more empirical format than the somewhat evocative, and arguably intuitive, violin plot. As a result, however, it makes it easy to compare the distribution of multiple datasets, which can be done within the comparisons tab.

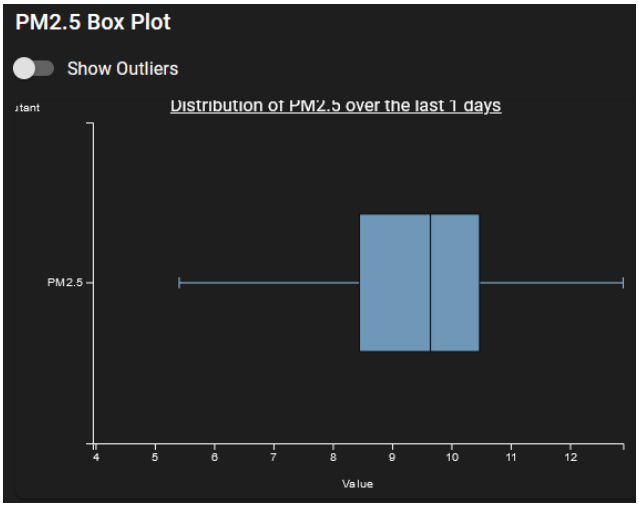

Figure 16. Box Plot: PM2.5 over the past 24-hours, outliers-removed

- Spike Map: Visualising geographical data, the spike map combines the intuitive nature of a high data-ink bar chart with an orange interpolated colour scheme to demonstrate the levels of each location, making for straightforward identification of hotspots, such as surrounding the airport. Each spike represents the average of that sensor over the time period selected. The spike heights are scaled logarithmically, to ensure a wide range of values are represented effectively without any large values dominating the visualisation, an issue established during the creation of the SCD. Emboldened for clarity, the spikes stand out on the light background map, courtesy of OpenStreetMap (ope). This choice of map colour was selected for its clarity over the colourful or satellite options, however, it somewhat clashes with the theme when dark mode is toggled, something that should be addressed in future. This map was implemented with D3 and leaflet and its interactive nature grants the user control over which regions they want to view and where they wish to zoom in for more detail, giving the data a greater sense of reality by laying it over the recognisable real-world locations, rather than just figures on a plot.

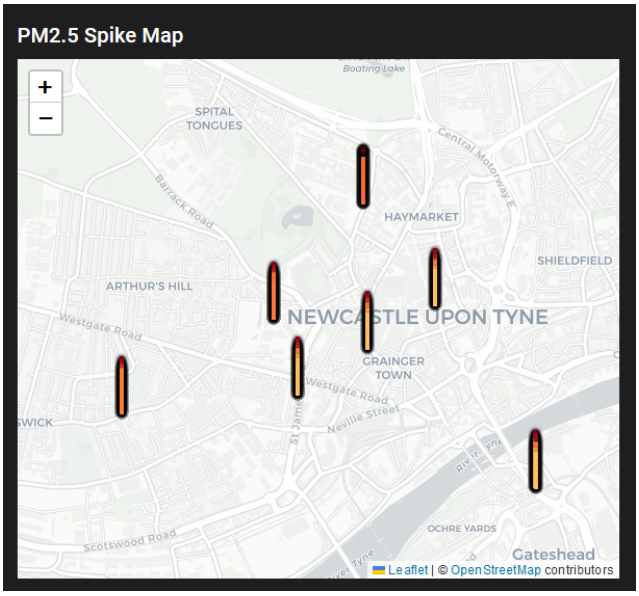

Figure 17.

- Bubble Map: Providing a slightly different visualisation to the spike map, the sizes of the data points scale outwardly to represent data magnitude. Similarly logarithmically scaled, the 'bubble' shape makes colour adjustments clear and emphasises the encompassing nature of air pollution. Bubbles feel like they are infringing upon the freedom of the map, in the same way air pollution restricts breathability and endangers public health.

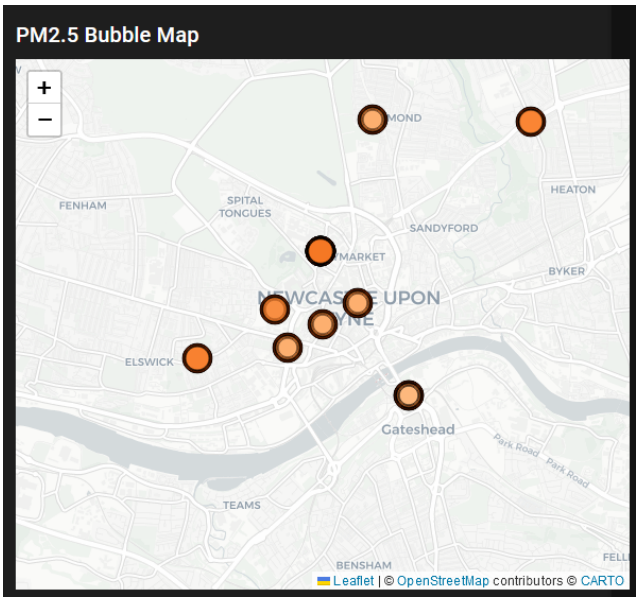

Figure 18.

- Gauges, a success from the previous project for providing immediacy (which is so sought by digital native generations) of information, are implemented on the right-hand side of the screen to visualise the current state of the environment. For PM2.5, PM10 and NO2, the 24-hour average is displayed with a blue-to-orange

gauge changing colour when the safe limit is being exceeded. Being generated from the MUI library, these suit the overall theme effectively, as well as avoiding the reds and greens which are less accessibility friendly, yet still playing on innate assumptions of 'safety' colours and 'danger' colours.

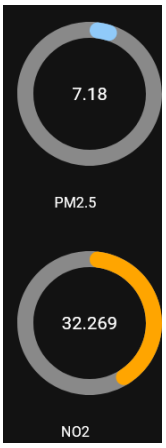

Figure 19. Gauges displaying PM2.5 and NO2

3.4 Integration

API Calls

Integration between the front and back end relies upon API calls to fetch the data as needed. To increase modularity and reduce complexity, this functionality was separated into a custom hook to be called upon whenever data is required. This, in turn, calls upon a service which, using the built-in `FETCH` API, sends the `GET` request via the back-end URL stored in `.env`, put together with the relevant endpoint and parameters.

The `useDataFetch` hook takes the endpoint and parameters as an argument, before asynchronously fetching the data. This is handled in the form of a central cache and a fetch queue, with the central cache. The fetch queue ensures that when called upon to fetch data by multiple sources at once, they are handled one after the other, first-in-first-out. When a request is reached, the central cache is checked to verify whether the data being requested is already present in the cache, if not, the data is fetched using the `dataService` before being stored in the central cache.

Real-time Updates

To keep in line with the goal of the programming working in real-time, while balancing application performance the following compromise was found: if two minutes have passed since an item reached the cache, then it will be ignored - prompting a refresh. 'Debouncing' is also implemented at 300ms, to prevent calls being made in quick succession and ensure there is time for the cache to update - improving overall performance. The result is that, on loading, all visualisations will appear to load simultaneously, with only the first item in the queue making the service call to fetch and cache the data.

When data is being fetched, `useState`s are used in the `useDataFetch` to communicate to the rest of the program that loading is taking place, or an error in the event that something goes wrong. When new data is successfully fetched, the `data` state is updated, triggering all components making use of this hook to re-render and thus ensuring all visualisations are up to date. When the `loading` state is `true`, all components are scripted to give feedback in the form of a circular loading spinner UI element, courtesy of the MUI library.

Cross-origin is enabled in the back-end to allow the programs to communicate across different ports without being blocked by the browser's default 'same-origin' policy. On receiving a GET request at the specified endpoints, the corresponding functions are called upon with the passed-in arguments, before SocketIO returns the expected data in JSON format with a status code of '200' to denote success, or '500' in the event of something going wrong.

3.5 Testing

Testing is critical to the successful development of any application, with UrbanUI being no different. Debugging during development was aided by the creation of an error boundary for displaying error messages within the UI, as well as extensive exception handling throughout the API. Although formalised iterative testing was not laid out, testing took place throughout development in the form of a fast and practical ad hoc approach. Though this speeds up production dramatically and can sometimes find bugs missed by formal testing, the pitfall with the lack of formal testing is the potential for gaps in testing coverage, and the irreproducability of tests. As such, modifications to existing code could cause unforeseen consequences and lead to a considerable time sink in debugging and recovery. With thorough unit tests, all aspects of code can be double-checked upon making changes - reducing unexpected bugs dramatically, particularly in unforeseen situations.

Regardless, ad hoc testing has its place. During the early development of a one-person project, thorough unit tests, formalised iterative design and the overheads that come with them can be forgone in favour of putting together the basics, as has been achieved in this application. However, before any future development takes place, it would be prudent to implement unit tests and lay out a formal plan for testing moving forward. This will ensure the application remains robust and maintainable as it scales and evolves. Usability testing through user feedback sessions should also be implemented as part of this, to generate qualitative data about usability to guide the future of the dashboard. Performance testing could also take place to evaluate the responsiveness and speed of the dashboard under different conditions, helping to highlight areas that require improvements to their efficiency and fail to meet the objective of a powerful tool capable of competing for attention span in a data-dense world.

4 Evaluation

The project was successful in developing a 'one-page' dashboard complete with various visualisations of local environmental data, being fetched, processed and sanitised in real-time.

Effective UI Design

The developed UI is created in Material design's image, implementing popular widgets and design choices including tabs, toggles and the burger menu. The outcome is an intuitive yet comprehensive dashboard for examining local environmental data.

Data Fetching and Aggregation

UrbanAPI successfully fetched data from the UO's API and provided an effective interface for the front end to fetch from. Aggregation to a 15-minute median grants greater control over the data. Though downsampling to a mean is used on the data regardless, the choice to do so is made on command within the UrbanAPI app script and can be altered or removed with ease as visualisations require it. Downsampling allowed for the generation of clearer visualisations whilst not too heavily dampening the shape of the overall data due to the nature of the data not being too offset by a wide spread of values regardless.

Visualisations

Visualisations were included for trend identification, distribution analysis, geographical data and time series forecasting. The scatter plot could be improved to include an option for multi-variate comparison on a single chart, removing the need for multiple scatter graphs and enabling easier comparisons to be made and playing into one of the innate benefits of the scatter as a format. Colour schemes are clear and adjust as necessary to suit both light and dark modes while maintaining a sense of consistency in both.

The inclusion of distribution plots, both violin and box, was a particularly effective inclusion for providing a wide range of tools for interpretation and analysis, showing not only average trends over time but how the city is faring as a whole. Both violin and box plots provide insight into how the data is spread and its central tendency, particularly due to the inclusion of a check-box widget for toggling the inclusion of outliers.

The maps grant legitimacy to the data by drawing attention to its real-world nature and successfully combining sensor data with their appropriate geographic locations. The spike map could be improved upon by adjusting the scaling of the spikes, perhaps with linear values or adjustments to the logarithmic scale, to ensure values are easy to differentiate. In a similar vein, a legend should be implemented to give clearer meaning to the spike sizes. Though this would detract from the simplicity, it could be implemented in such a way that it is subtle and doesn't hinder the aesthetics of the maps.

As discussed, the forecasting model could be improved upon with a larger set of training data running at regular intervals throughout the day. Forecasting need not be real-time as it doesn't vary much from minute to minute. The decision to make this real-time in line with the design of the rest of the visualisations is something that greatly hinders the usability of the application and should be altered moving forward.

Interactivity

Tooltips were implemented on the line and scatter plots, successfully providing the user with 'drill-down data' on request. On the line graph, this is accompanied by a line along the axis where the cursor is, allowing for ease of scrolling along the data and viewing the values over time. Adjustable time ranges via the drop-down menu in the top-right were similarly successful at providing 'drill-down', as the default time period of a single day immediately provides the most relevant yet simplest datasets. For more information, the user need only interact with the drop-down and alter to one of the other periods available, including 3 days, 1 week and 1 month. Tabs enabled intuitive scrolling between different variables for the same set of visualisations to immediately snap to a new sensor type. The short load time could perhaps be reduced by caching other tabs while another is being viewed, however, it is unnecessary to load in so many datasets, especially when these could vary across several variables and numerous time scales. The user is empowered by the tabs at hand and yet no unnecessary visualisations are present, only being present if so desired, much like flicking through the pages of a book or data portfolio. Checkboxes to toggle outliers grants extra flexibility to those plots where this is particularly relevant, namely the distribution plots.

The burger menu brings further control in the name of categorisation. This has a positive impact on usability, as not only can the user seek additional information, they can reduce the clutter on the page as required for what they have in mind. In contrast, the comparisons tab brings the power to add as many variables as desired to the dashboard at once, even ordering them by graph type for the convenience of comparing and contrasting with ease. The level of control of information via these interactive functions is one of the greatest successes of this application as compared to the previous project.

Error handling and Robustness

Error boundaries and extensive exception handling presented a decent amount of error handling. As discussed previously, unit testing remains a requirement for the project going forward, but through informal testing of all the elements of the application it is without obvious errors or crashes - and when there were crashes previously, all were caught and provided an informative UI describing the problem. This is important as it allows users to report their crashes and errors to support as necessary. The fallback mechanism of backup CSV files in the event of UO downtime is a great success that was adopted from the previous project and continues to be

critical to the robustness of the application, ensuring data persistence even when the primary data source is not available.

Accessibility

Colour schemes are based on the 'ColorBrewer' tool and avoid the problematic red-green and rainbow schemes entirely. Examples of colour schemes employed include blue-orange, yellow-orange, and purple-orange. Orange was employed liberally as a substitute for red but would still evoke feelings of danger. The light-dark toggle was successfully implemented, with the app defaulting to the default of the system it's running in. Text is in the Roboto font in line with MUI's default and text size scales with the page, but is a size that could very reasonably be deemed readable. Alt-text is provided for those who require it, however, as discussed this is problematic on a web page full of visualisations, however, text-based summaries of graphs could be implemented in future as improvements to AI make it a more viable option for real-time visual interpretations.

Deployment and Containerisation

Despite the creation of a fully functioning web application, deployment is still yet to be implemented. This would require a hosting environment such as AWS or Azure and could be set up with a virtual machine running as the server. The increase in complexity for implementation of a server, with back-end deployment and potentially SQL databases meant it was deemed out of scope for this project - which focuses primarily on visualisation and applications of the sensors that make up a smart city. Similarly, Docker containerisation was considered to ensure consistency across different environments and to facilitate the development of a more formalised Continuous Integration and Continuous Deployment (CI/CD) pipeline, but the necessary time investment was deemed likely to detract from the primary focuses of this project. These are still developments that could be considered in the future of this project.

Multi-device Support

The aim of effective mobile support was not met and would require more careful consideration of UI design. Though the page functions on mobile, it does not suit the platform due to the vast layout changes necessary. One possible solution would be to create a separate, optimised version of the web page specifically for mobile devices, featuring a simplified interface and improvements to the auto-resize components and settings. Despite considerable effort being placed on ensuring that the cards in the main dashboard grid used percentages for sizing and locations over firm pixel counts, in addition to the auto-resize component, the drastic change necessary for functioning on mobile is not something that can be achieved without more careful consideration, design and thorough testing.

5 Conclusions and Further Work

This project successfully developed UrbanUI, a responsive and intuitive environmental dashboard offering accessible and actionable air quality data to users regardless of their technical knowledge level. Building upon the successes of a similar project, data was integrated from the Newcastle Urban Observatory API into a user-friendly interface built with React and MaterialUI, resting upon a robust Flask back-end for data-fetching and preprocessing.

The accomplishments of development include several key features. The MaterialUI-built interface resulted in a clean and intuitive design, with both light and dark themes for accessibility and modernisation. Implementation of interactive elements such as toggles, tabs and the burger menu made significant advancements in user experience over the previous project and placed effective navigation and user engagement at the heart of the application. Further, the effortlessly integrated Flask back-end effectively fetches and aggregates data, with backup mechanisms in place to ensure data integrity and persistence even in the event of downtime.

Visualisations include line graphs, scatter plots, violin plots and box plots, as well as spike and bubble maps for evocative geographical information. On top of offering a comprehensive insight into environmental data, control widgets were implemented to provide extensive user control over data granularity, granting progressive data disclosure or 'drill-down' features to retain accessibility and prevent data overload. Colour schemes were considered carefully to ensure accessibility by those with colour-vision deficiency, a symbol of UrbanUI's philosophy of accessibility.

Despite these successes, several limitations were identified. Mobile optimisation is a feature which, considering the percentage of users who surf the web on their phones, should be considered first and foremost as a goal for future development. Working with sizing and managing formatting marks one of the primary difficulties faced during development, and as such development of a mobile-optimised web page represents a great learning opportunity. Future additions to the app should also follow a formalised testing plan, featuring unit and integration tests to provide a more robust development process than the ad hoc testing procedures which contributed to the conception of the app.

Looking forward, several steps should be taken to ensure proper deployment and containerisation, especially if the code is intended for public use. The establishment of a Continuous Integration and Continuous Deployment (CI/CD) pipeline would allow for automated testing and deployment with greater consistency and reliability, as well as simplifying of updates. Docker containerisation would work to ensure consistency across all development and production environments and would be an effective way of implementing automatic builds and deployment in line with CI/CD.

Further, the forecasting model should be improved upon with server-side caching. This will allow for larger datasets and more accurate predictions. Mean absolute

error could be generated once per day for general guidance of the forecast accuracy and hourly forecasts could be generated without the need for splitting training and testing data. In addition to accuracy, this would also improve user experience by reducing the processing time incurred by the real-time demand of forecasting, a labour-intensive act in itself.

User feedback is something of a prerequisite of effective iterative testing post-release, as such the implementation of a survey screen would help to generate qualitative reports regarding successes and potential improvement. This could particularly focus on the perceived complexity and aesthetic quality of the visualisations, as the complexity of the D3 library and the unreliable nature of MUI X at times was a major barrier during development - especially due to the smaller amounts of documentation as compared to other libraries. In addition to this, retaining subjectivity when working with these graphs is innately difficult when becoming so familiar not only with the format but the aesthetics. A feedback survey could include screenshots of the visualisations created with different libraries to ascertain public opinion on the many options available.

In conclusion, UrbanUI has succeeded in building the foundations of a comprehensive environmental dashboard to empower users with real-time air quality data. The identified limitations will lay the groundwork for future development so that UrbanUI can evolve into a robust, user-friendly and widely accessible tool. This project has demonstrated how with empathetic design, advanced data visualisation techniques can work with real-time data processing to improve both public understanding and engagement with local environmental data.

6 Acknowledgements

Extending gratitude to the Newcastle Urban Observatory team for providing public access to the data from their extensive sensor network, and without whom this project would not be possible.

References

Chart.js: Simple yet flexible javascript charting. https://www.chartjs.org/. Accessed: May 17, 2024.

D3.js: Data-driven documents. https://d3js.org/. Accessed: May 17, 2024.

Material-UI: React components for faster and easier web development. https://mui.com/material-ui/. Accessed: May 17, 2024.

Openstreetmap. https://www.openstreetmap.org/#map=6/54.910/-3.432. Accessed: 22nd May 2024.

Assessment of urban growth in relation to urban sprawl using landscape metrics and shannon's entropy model in jalpaiguri urban agglomeration, west bengal, india. *Geocarto International*, 39(1):2306258,

2024. doi: 10.1080/10106049.2024.2306258. URL https://doi.org/10.1080/10106049.2024.2306258.

Acoem UK. Urban observatory monitors newcastle with new generation of environmental sensors. *Acoem UK News*, 2024. URL https://www.acoem.com/united-kingdom/news/urban-observatory-monitors-newcastle-with-new-generation-of-environmental-sensors/. Accessed: 2024-05-24.

U. Air. Defra uk air information resource. https://uk-air.defra.gov.uk/, 2024. Accessed: 17 May 2024.

Bloomberg. How bloomberg terminal ux designers conceal complexity, December 2022. URL https://www.bloomberg.com/company/stories/how-bloomberg-terminal-ux-designers-conceal-complexity/.

Bootstrap Contributors. Bootstrap, 2024. URL https://getbootstrap.com/.

J. Chicca and T. Shellenbarger. Connecting with generation z: Approaches in nursing education. *Teaching and Learning in Nursing*, 13(3):180–184, 2018. ISSN 1557-3087. doi: https://doi.org/10.1016/j.teln.2018.03.008. URL https://www.sciencedirect.com/science/article/pii/S1557308718300520.

ColorBrewer. ColorBrewer 2.0. https://colorbrewer2.org/. Accessed: 19th May 2024.

P. Cudlín, J. Seják, J. Pokorný, J. Albrechtová, O. Bastian, and M. Marek. Chapter 24 - forest ecosystem services under climate change and air pollution. In R. Matyssek, N. Clarke, P. Cudlin, T. Mikkelsen, J.-P. Tuovinen, G. Wieser, and E. Paoletti, editors, *Climate Change, Air Pollution and Global Challenges*, volume 13 of *Developments in Environmental Science*, pages 521–546. Elsevier, 2013. doi: https://doi.org/10.1016/B978-0-08-098349-3.00024-4. URL https://www.sciencedirect.com/science/article/pii/B9780080983493000244.

Department for Environmental Food and Rural Affairs. Air pollution statistics. https://www.gov.uk/government/statistics/air-quality-statistics/concentrations-of-particulate-matter-pm10-and-pm25, 2024. [Accessed: 20th May 2024].

H. Eisfeld and F. Kristallovich. *The Rise of Dark Mode: A qualitative study of an emerging user interface design trend*. Dissertation, 2020. URL https://urn.kb.se/resolve?urn=urn:nbn:se:hj:diva-50563.

A. Erickson, K. Kim, G. Bruder, and G. F. Welch. Effects of dark mode graphics on visual acuity and fatigue with virtual reality head-mounted displays. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*, pages 434–442, 2020. doi: 10.1109/VR46266.2020.00064.

R. Ewings. Smart city climate notebook. https://github.com/rufuste/SmartCityClimate/blob/master/Notebooks/37.ipynb, 2023. Created: May 24, 2023.

Facebook. Prophet: A forecasting tool for time series data in python. https://facebook.github.io/prophet/. Accessed: May 17, 2024.

Facebook. React documentation, 2024. URL https://react.dev/. Accessed on

17th May, 2024.

L. Jin, Y. Liu, B. Dong, Y. Wang, and Y. Zhang. Deep learning in electronic health records for cardiovascular event prediction: a systematic review. 2021. URL https://arxiv.org/abs/2103.12910.

A. Korah, J. A. Koch, and M. C. Wimberly. Understanding urban growth modeling in africa: Dynamics, drivers, and challenges. *Cities*, 146:104734, 2024. ISSN 0264-2751. doi: https://doi.org/10.1016/j.cities.2023.104734. URL https://www.sciencedirect.com/science/article/pii/S0264275123005462.

Leaflet.js. Leaflet api reference, Accessed: 2024. URL https://leafletjs.com/reference.html.

C. Li, X. Xia, Y. Lin, and H. Wen. Polluted cognition: The effect of air pollution on online purchasing behavior. *Journal of Retailing and Consumer Services*, 79:103839, 2024. ISSN 0969-6989. doi: https://doi.org/10.1016/j.jretconser.2024.103839. URL https://www.sciencedirect.com/science/article/pii/S0969698924001358.

S. Ma, C. Chen, H. Khalajzadeh, and J. Grundy. A first look at dark mode in real-world android apps. *ACM Trans. Softw. Eng. Methodol.*, 33(1), nov 2023. ISSN 1049-331X. doi: 10.1145/3604607. URL https://doi.org/10.1145/3604607.

I. Manisalidis, E. Stavropoulou, A. Stavropoulos, and E. Bezirtzoglou. Environmental and health impacts of air pollution: A review. *Frontiers in Public Health*, 8:14, 2020. doi: 10.3389/fpubh.2020.00014.

K. McGurgan, E. Fedoroksaya, T. M. Sutton, and A. M. Herbert. Graph design: The data-ink ratio and expert users. In *VISIGRAPP*, 2021. URL https://api.semanticscholar.org/CorpusID:232094223.

K. Moreland. Diverging color maps for scientific visualization (expanded). 2016. URL https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.880.9033&rep=rep1&type=pdf.

National Eye Institute. Types of color vision deficiency, Accessed: 2024. URL https://www.nei.nih.gov/learn-about-eye-health/eye-conditions-and-diseases/color-blindness/types-color-vision-deficiency.

N. U. Observatory. Urban observatory api examples. Newcastle Urban Observatory API documentation, 2024. URL https://newcastle.urbanobservatory.ac.uk/api_docs/examples/example/2/. Accessed: May 15, 2024.

Our World in Data. The majority of people in the world now live in cities. https://ourworldindata.org/data-insights/the-majority-of-people-in-the-world-now-live-in-cities, 2024. Accessed: May 15, 2024. Data source: European Commission, 2023.

Pallets. Flask documentation. https://flask.palletsprojects.com/, 2024. Accessed: May 17th, 2024.

O. Pastushenko, J. Hynek, and T. Hruška. Evaluation of user interface design metrics by generating realistic-looking dashboard samples. *Expert Systems*, 38(5), Sept. 2019. ISSN 1468-0394. doi: 10.1111/exsy.12434. URL http://dx.doi.org/10.1111/exsy.12434.

R. Pawanker. Climate change, air pollution, and biodiversity in asia pacific: impact on allergic diseases. *Asia Pacific Allergy*, 9(1):e11, 2019. doi: 10.5415/apallergy.2019.9.e11.

D. Pumain. *Urban Systems Dynamics, Urban Growth and Scaling Laws: The Question of Ergodicity*, pages 91–103. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012. ISBN 978-3-642-24544-2. doi: 10.1007/978-3-642-24544-2_6. URL https://doi.org/10.1007/978-3-642-24544-2_6.

A. Rashid. The role of responsive design in web development. *Webology*, 14(1), 2017. URL https://www.webology.org/data-cms/articles/20200515031209pma157.pdf.

T. Roelandts. The diverging color map is the new rainbow color map. 2020. URL https://tomroelandts.com/articles/the-diverging-color-map-is-the-new-rainbow-color-map.

L. Sabieddine. Reclaiming my attention span: My week without tiktok. *UWIRE Text*, page 1, Mar. 2023. URL https://link.gale.com/apps/doc/A743150355/AONE?u=anon~4932e9ad&sid=googleScholar&xid=d9ab622d. Accessed: 20 May 2024.

D. J. Severtson and J. D. Myers. The influence of uncertain map features on risk beliefs and perceived ambiguity for maps of modeled cancer risk from air pollution. *Risk Analysis*, 33(5):818–837, Sept. 2012. ISSN 1539-6924. doi: 10.1111/j.1539-6924.2012.01893.x. URL http://dx.doi.org/10.1111/j.1539-6924.2012.01893.x.

A. I. Tiotiu, P. Novakova, D. Nedeva, H. J. Chong-Neto, S. Novakova, P. Steiropoulos, and K. Kowal. Impact of air pollution on asthma outcomes. *International Journal of Environmental Research and Public Health*, 17(17):6212, August 2020. doi: 10.3390/ijerph17176212.

D. Todorovic. Gestalt principles. *Scholarpedia*, 3(12):5345, 2008. ISSN 1941-6016. doi: 10.4249/scholarpedia.5345. URL http://dx.doi.org/10.4249/scholarpedia.5345.

UK Government. Understanding wcag, Accessed: 2024. URL https://www.gov.uk/service-manual/helping-people-to-use-your-service/understanding-wcag.

A. Van Cauwenberge, G. Schaap, and R. van Roy. "tv no longer commands our full attention": Effects of second-screen viewing and task relevance on cognitive load and learning from news. *Computers in Human Behavior*, 38:100–109, Sept. 2014. ISSN 0747-5632. doi: 10.1016/j.chb.2014.05.021. URL http://dx.doi.org/10.1016/j.chb.2014.05.021.

C. Ware, M. Stone, and D. A. Szafir. Rainbow colormaps are not all bad. *IEEE Computer Graphics and Applications*, 43(3):88–93, May 2023. ISSN 1558-1756. doi: 10.1109/mcg.2023.3246111. URL http://dx.doi.org/10.1109/MCG.2023.3246111.

O. M. Yigitbasioglu and O. Velcu. A review of dashboards in performance management: Implications for design and research. *International Journal of Accounting*

Information Systems, 13(1):41–59, Mar. 2012. ISSN 1467-0895. doi: 10.1016/j.accinf.2011.08.002. URL http://dx.doi.org/10.1016/j.accinf.2011.08.002.

Y. Zhang, Q. Sun, J. Liu, and O. Petrosian. Long-term forecasting of air pollution particulate matter (pm2.5) and analysis of influencing factors. *Sustainability*, 16 (1):19, 2024. doi: 10.3390/su16010019. URL https://www.mdpi.com/2071-1050/16/1/19.